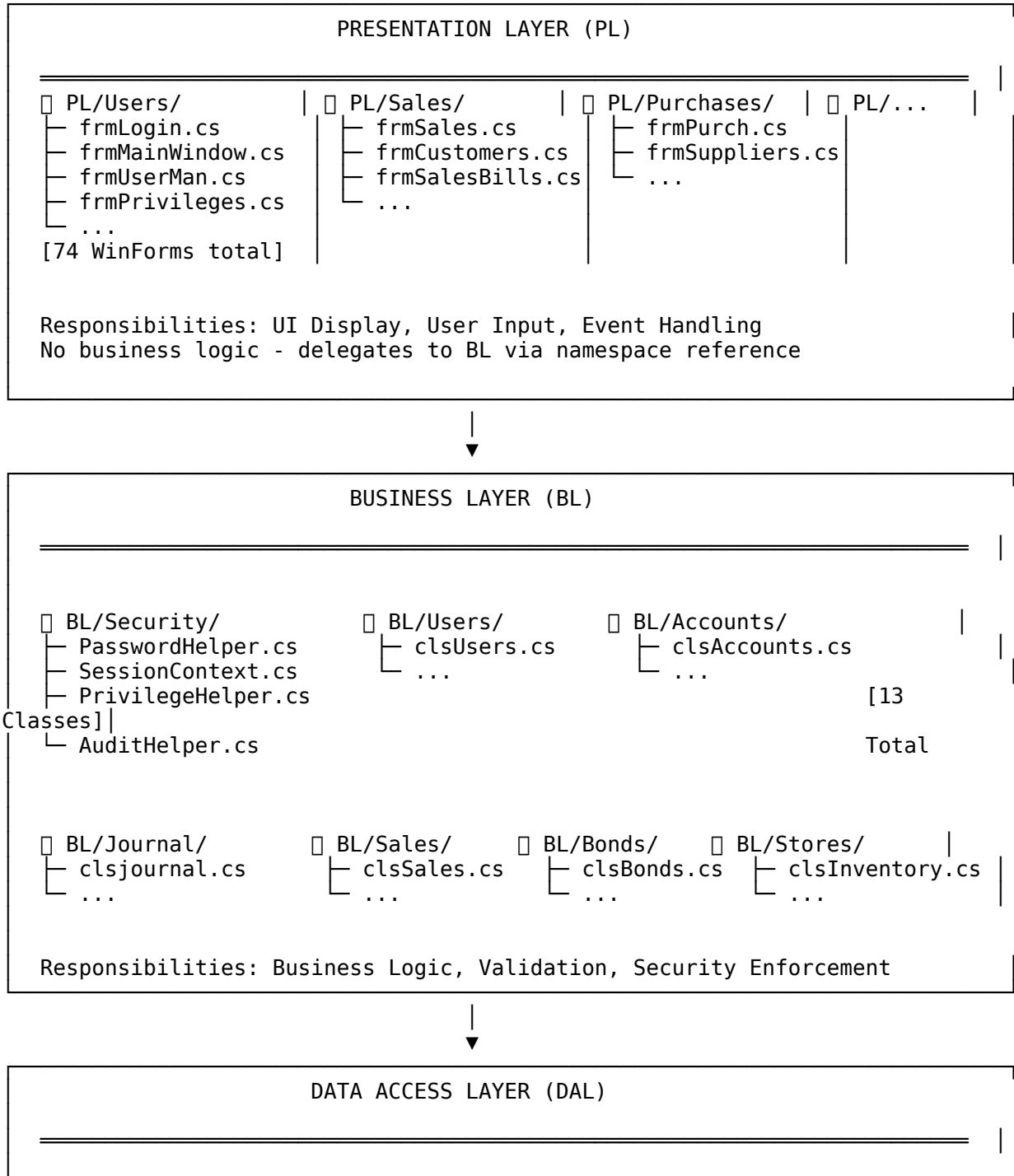


CMP - Comprehensive Project Map
نظام الحسابات المتكامل | IntegratedAccountsSystem

1. البنية المعمارية الكاملة | Full Architecture

...

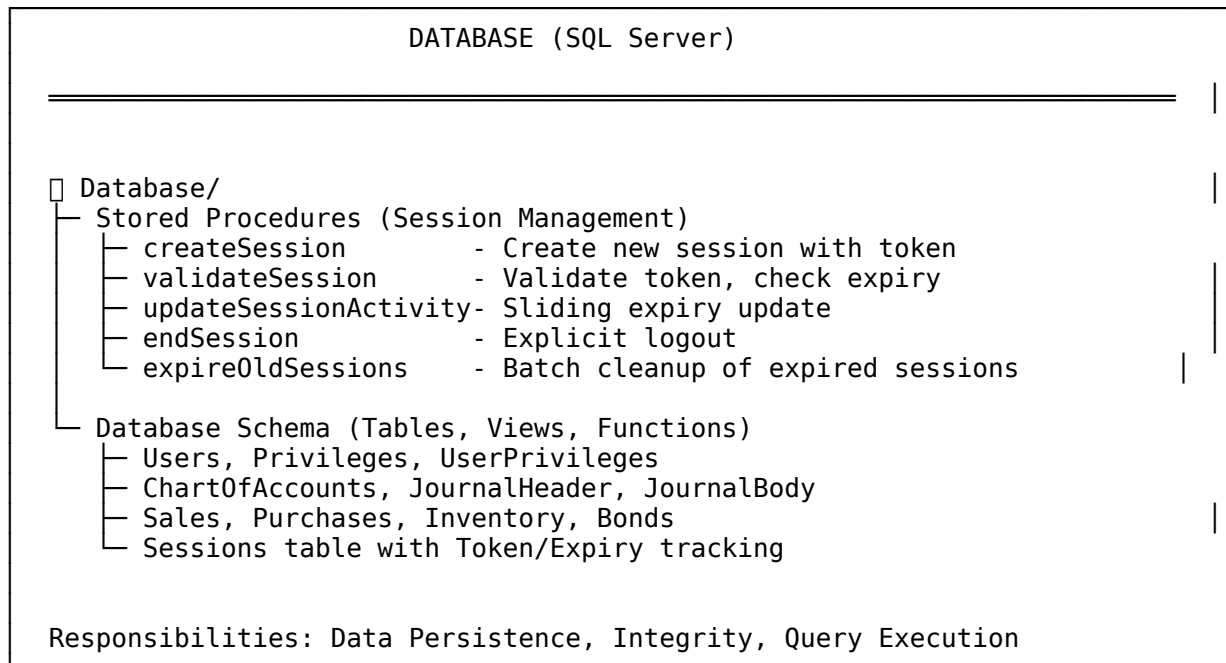


```

DAL/
├── clsCN.cs (187 lines)
│   ├── ExecuteNonQuery()
│   ├── ExecuteReader()
│   └── GetDataTable()
│
│   ├── ValidateStoredProcedureCall() → SQL Injection Prevention
│   └── Transaction Support (BeginTransaction/Commit/Rollback)
└──

```

Responsibilities: SQL Execution, Connection Management, SP Validation

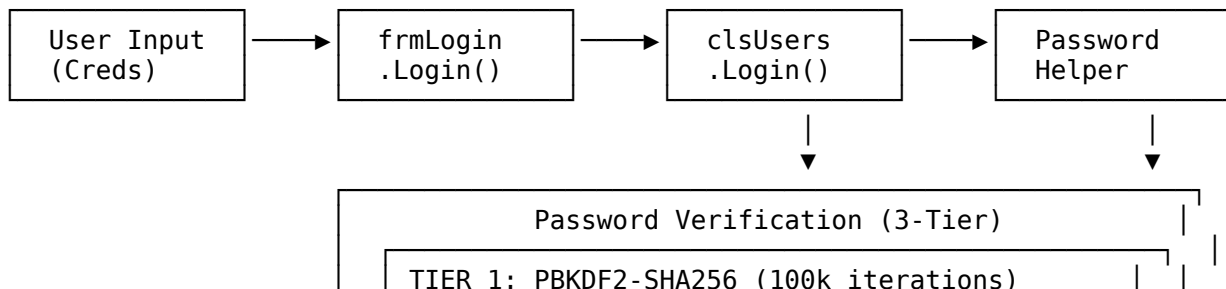


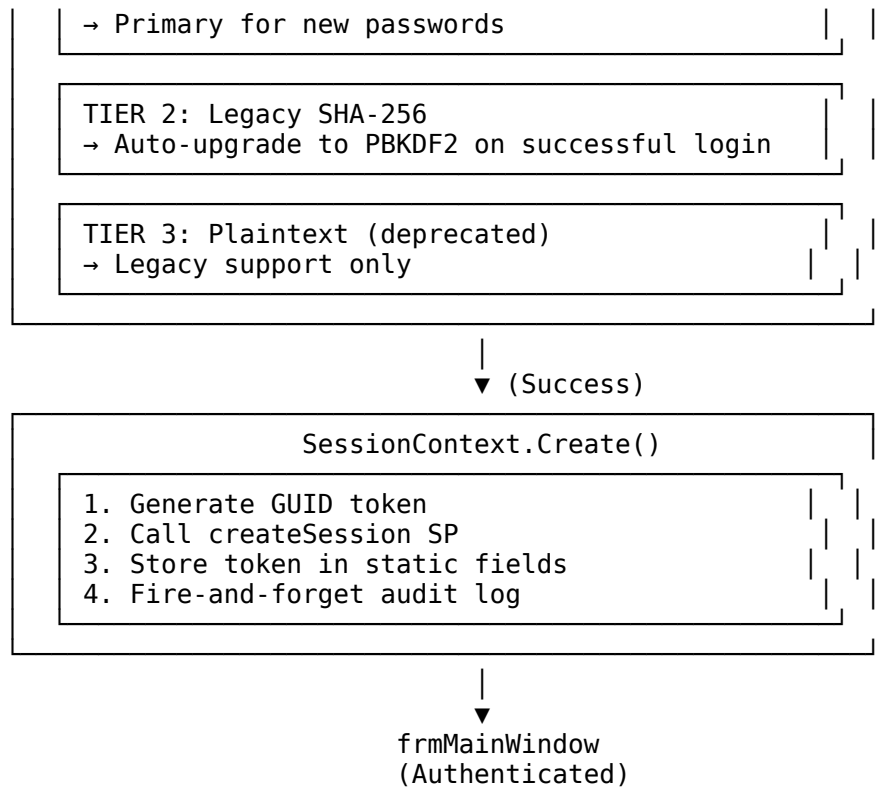
...

2. تدفق البيانات والعمليات | Data & Process Flows

2.1 تدفق المصادقة والتحقق | Authentication Flow

...

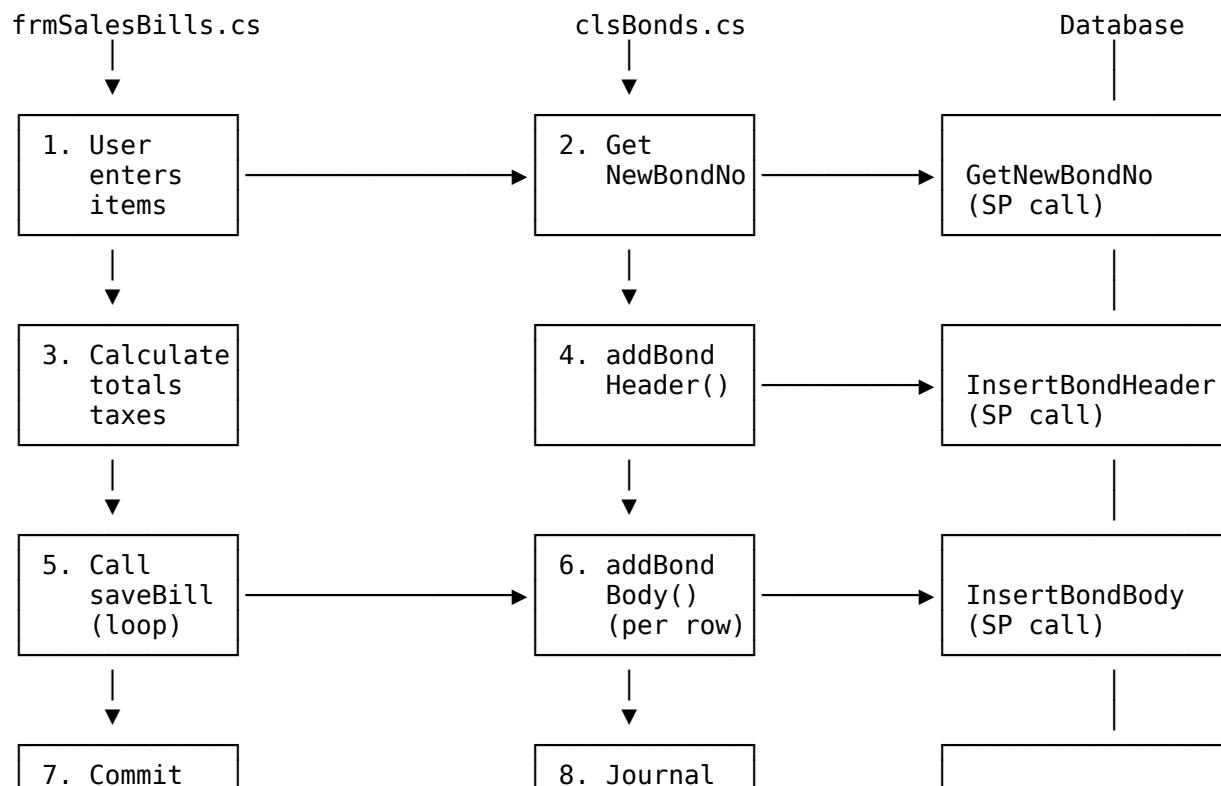


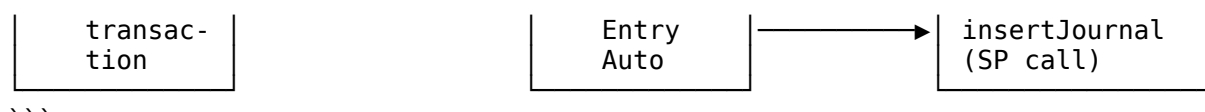


...

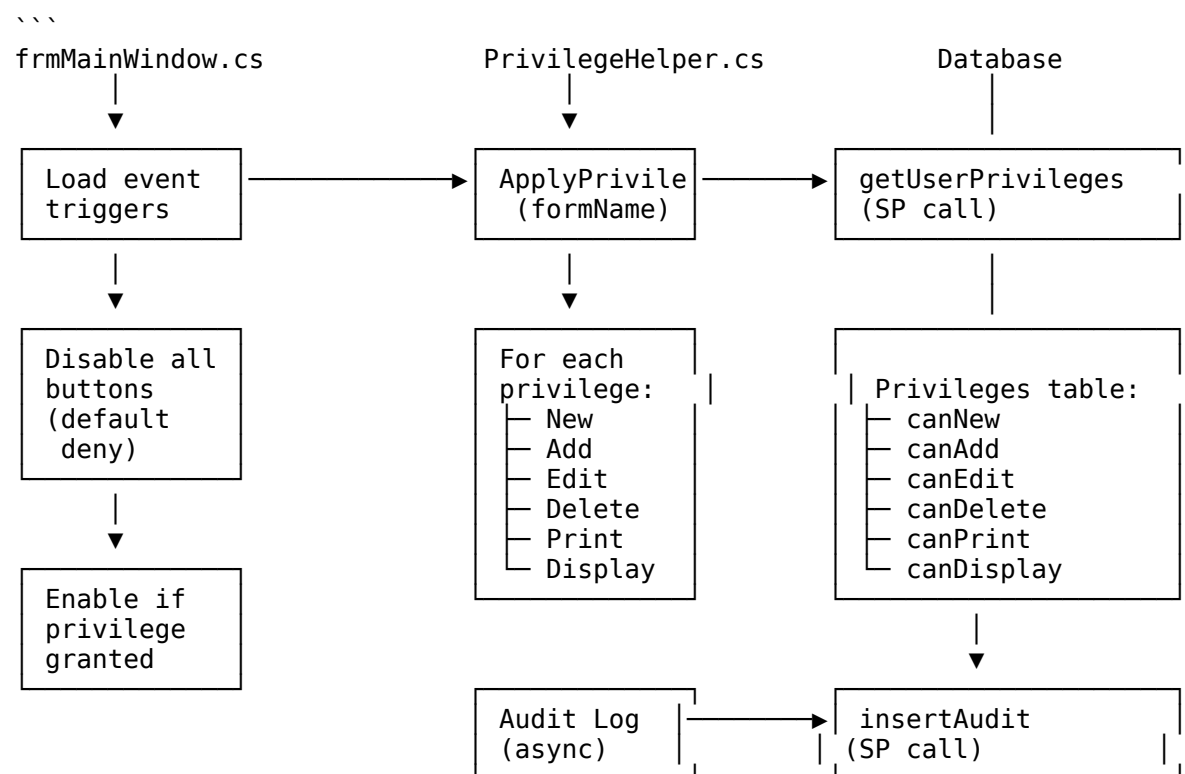
2.2 تدفق عملية البيع | Sales Bill Flow

...



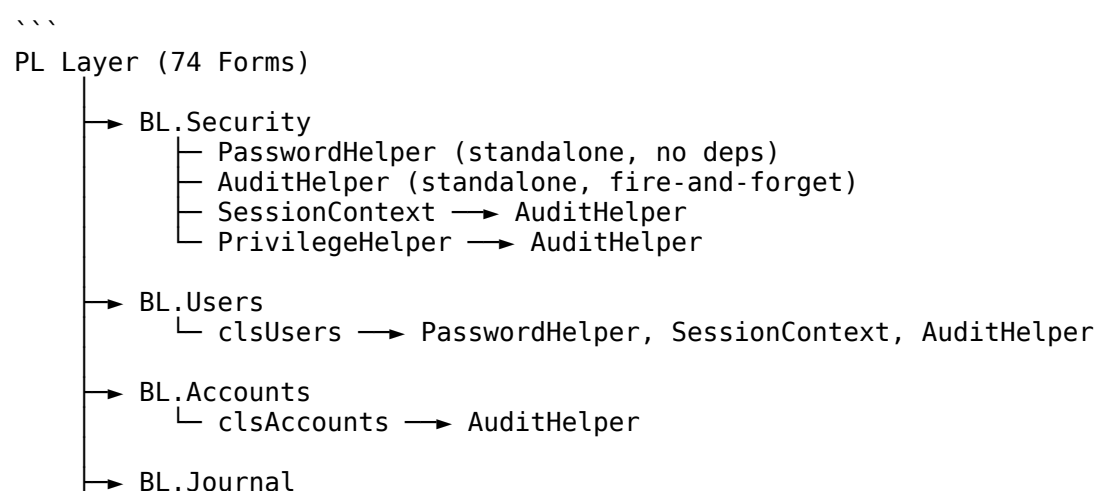


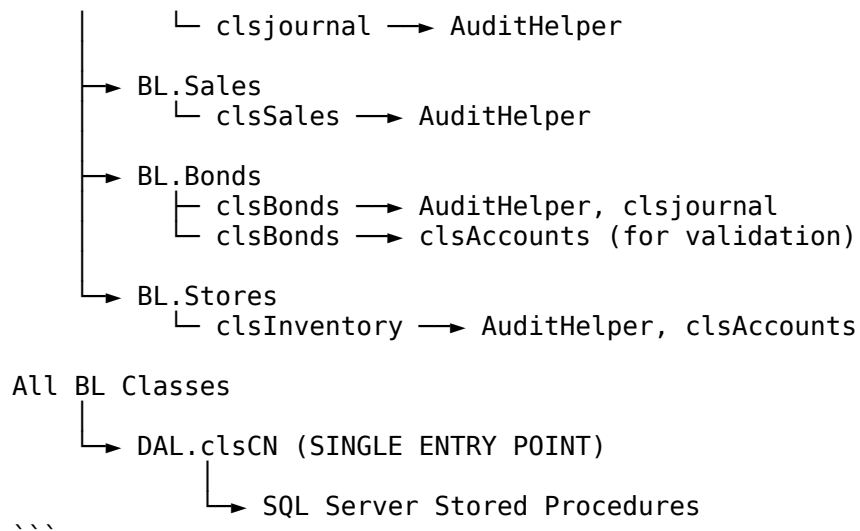
2.3 | تدفق نظام الصلاحيات Privilege System Flow



3. | العلاقات والاعتماديات Relationships & Dependencies

3.1 Dependency Graph



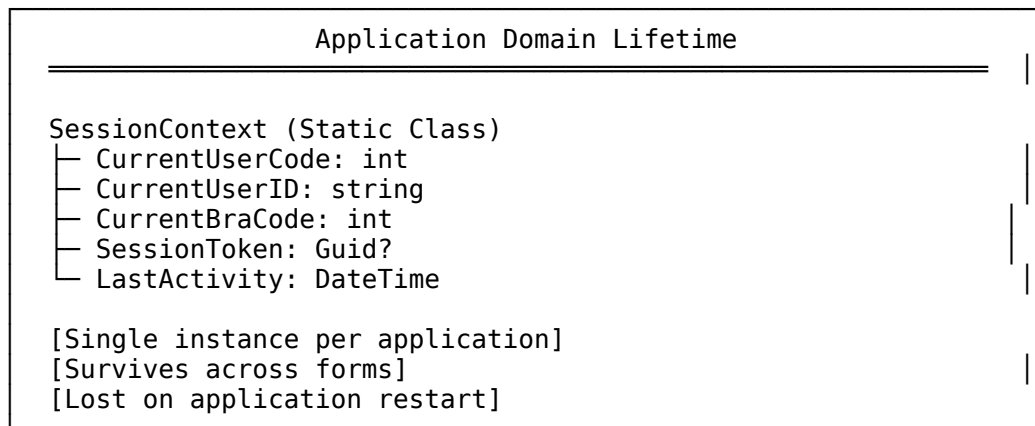


3.2 Cross-Layer Communication

From Layer	To Layer	Method	Purpose
PL.Forms	BL	`new clsXXX()`	Instantiate business class
PL.Forms	BL	`classInstance.method()`	Call business logic
BL	DAL	`new clsCN()`	Database connection
BL	DAL	`clsCN.GetDataTable()`	Query execution
DAL	DB	SP calls	Data persistence

3.3 Static State Dependencies

...



...

4. مسارات التنفيذ وسير العمل | Execution Paths

4.1 Application Startup Path

...

Program.Main()

```

    ↓
ApplicationConfiguration.Initialize()
    ↓
Application.Run(new frmLogin())
    ↓
[User sees login screen - app waits for input]
\\

```

4.2 Login Success Path

```

\\
frmLogin.btnLogin_Click()
    ↓
    → txtBranch.Text, txtUser.Text, txtPWD.Text
    ↓
clsUsers.Login(branch, userID, password)
    ↓
    → Query DB for user record
    → Verify password (3-tier)
    → Return DataTable with user info
    ↓
    (if rows > 0)
    ↓
SessionContext.Create(userCode, userID, braCode)
    ↓
    → new clsCN()
    → new SqlParameter[5]
    → cmd.Parameters.AddRange(allPara)
    → cmd.ExecuteNonQuery()
    → SessionContext fields populated
    → AuditHelper.LogSessionCreated() [fire-and-forget]
    ↓
frmMainWindow fmw = new frmMainWindow()
    ↓
fmw.Show()
    ↓
this.Hide()
\\

```

4.3 Sales Bill Save Path

```

    ...
frmSalesBills.btnSave_Click()
    |
    |→ Validate inputs
    |→ Calculate totals
    |
    ▼
clsBonds bonds = new clsBonds()
    |
    |→ bonds.GetNewBondNo(braCode, bondType)
    |   ↳ DAL → GetNewBondNo SP
    |
    |→ bonds.addBondHeader(...)
    |   ↳ DAL → InsertBondHeader SP
    |
    |→ For each item row:
    |   ↳ bonds.addBondBody(...)
    |       ↳ DAL → InsertBondBody SP
    |
    |→ bonds.addJournalHeader(...)
    |   ↳ DAL → insertJournalHeader SP
    |
    |→ For each account affected:
    |   ↳ bonds.addJournalBody(...)
    |       ↳ DAL → insertJournalBody SP
    |
    |→ AuditHelper.LogBillCreated() [fire-and-forget]
    |
    ▼
MessageBox.Show("تم الحفظ بنجاح")
    |
    ▼
Clear form / Refresh
    ...

```

4.4 Session Validation Path (On each form load)

```

    ...
frmXXX_Load()
    |
    ▼
if (SessionContext.SessionToken == null)
    |
    |→ MessageBox.Show("انتهت الجلسة")
    |→ Application.Exit()
    |
    ▼
clsCN.ValidateSession(SessionContext.SessionToken)
    |
    |→ Call validateSession SP
    |
    |→ Check result (valid/expired)
    |
    |→ If expired:
    |   ↳ SessionContext.Clear()

```

```

    └─ MessageBox.Show("انتهت الجلسة")
    └─ Application.Exit()
...

```

5. نقاط القوة والمخاطر والاختناقات | Strengths, Risks, Bottlenecks

5.1 نقاط القوة | Strengths

#	Strength	Description	Impact
1	**PBKDF2-SHA256**	100,000 iterations for password hashing	OWASP 2023 compliant, high security
2	**SQL Injection Prevention**	Regex validation in DAL.ValidateStoredProcureCall()	Prevents all known SQL injection vectors
3	**Token-based Sessions**	GUID tokens with 1-hour sliding expiry	Secure, stateless, scalable
4	**Async Audit Logging**	Fire-and-forget pattern with Task.Run()	Non-blocking, no performance impact
5	**Privilege System**	6-permission model per screen	Fine-grained access control
6	**Double-entry Accounting**	Journal header/body structure	Financial integrity
7	**Multi-tenant Support**	braCode in all operations	Branch isolation
8	**3-Tier Architecture**	Clean separation PL/BL/DAL	Maintainable, testable

5.2 المخاطر | Risks

#	Risk	Severity	Mitigation
1	**Static Session State**	High	Hard to test, memory leaks possible
2	**Plaintext Password Tier**	Medium	Legacy support only, logs warning
3	**No Encryption in Transit**	High	Add TLS to SQL connection
4	**Session Token in URL**	Medium	Ensure POST-only for sensitive ops
5	**No Rate Limiting**	Medium	Add login attempt throttling
6	**Sync Call for ValidateSession**	Medium	Consider async validation
7	**No Parameterized Queries in SP**	Low	SPs use parameters, safe
8	**Centralized Exception Handling**	Low	Add global exception handler

5.3 الاختناقات التقنية | Bottlenecks

#	Bottleneck	Location	Type	Recommendation
1	**DAL Connection per call**	clsCN	Performance	Implement connection pooling
2	**Sync validation on each form**	SessionContext	Latency	Cache validation, async refresh
3	**No lazy loading**	All forms	Memory	Implement virtual scrolling for grids
4	**Large DataTable returns**	getListOfAccounts, getAllCustomers	Memory	Implement pagination
5	**No caching layer**	All BL classes	Performance	Add Redis/MemoryCache

6. فرص التحسين | Improvement Opportunities

6.1 Critical Improvements (Priority 1)

...

1. Add Connection Pooling to clsCN

Current:

```
private SqlConnection GetConnection()
{
    SqlConnection cn = new SqlConnection(connectionString);
    cn.Open(); // New connection every time
    return cn;
}
```

Improved:

```
private static readonly object _lock = new object();
private static SqlConnection _cachedConnection;
private static DateTime _lastConnectionTime;
private const int ConnectionTimeoutMinutes = 5;

private SqlConnection GetConnection()
{
    lock (_lock)
    {
        if (_cachedConnection != null &&
            _cachedConnection.State == ConnectionState.Open &&
            (DateTime.Now - _lastConnectionTime).TotalMinutes <
            ConnectionTimeoutMinutes)
            return _cachedConnection;

        _cachedConnection?.Dispose();

        _cachedConnection = new SqlConnection(connectionString);
        _cachedConnection.Open();

        _lastConnectionTime = DateTime.Now;
        return _cachedConnection;
    }
}
```

2. Add Global Exception Handler

In Program.cs:

```
Application.ThreadException += (s, e) =>
{
    AuditHelper.LogException(e.Exception);
    MessageBox.Show($"حدث خطأ غير متوقع: {e.Exception.Message}",
        "خطأ", MessageBoxButtons.OK, MessageBoxIcon.Error);
};

AppDomain.CurrentDomain.UnhandledException += (s, e) =>
{
    var ex = e.ExceptionObject as Exception;
    AuditHelper.LogException(ex);
};
```

3. Add Login Rate Limiting

In Database:

```
CREATE TABLE LoginAttempts (
    AttemptID INT IDENTITY PRIMARY KEY,
    UserID NVARCHAR(15),
    IPAddress NVARCHAR(50),
    AttemptTime DATETIME,
    Success BIT
);
```

```
In clsUsers.Login():
var attempts= cn.GetDataTable("checkRecentAttempts",
```

```

        new SqlParameter("@userID", userID),
        new SqlParameter("@windowMinutes", 15));

if (attempts.Rows.Count >= 5)
    throw new AccountLockedException("تم تجاوز محاولات تسجيل الدخول");

```

...

6.2 High Priority Improvements (Priority 2)

...

4. Implement Async Session Validation

```

public static async Task<bool> ValidateSessionAsync()
{
    if (SessionToken == null) return false;

    return await Task.Run(() => {
        try {
            clsCN cn = new clsCN();
            var result = cn.GetDataTable("validateSession",
                new SqlParameter("@sessionToken", SessionToken.Value));
            return result.Rows.Count > 0;
        }
        catch { return false; }
    });
}

```

Usage:

```

private async void frmXXX_Load(object sender, EventArgs e)
{
    bool isValid = await SessionContext.ValidateSessionAsync();
    if (!isValid) { /* redirect to login */ }
}

```

5. Add Data Pagination for Large Result Sets

In clsAccounts:

```
public DataTable getAccountsPage(int braCode, int page, int pageSize)
{
    clsCN cn = new clsCN();
    return cn.GetDataTable("getAccountsPaginated",
        new SqlParameter("@braCode", braCode),
        new SqlParameter("@page", page),
        new SqlParameter("@pageSize", pageSize));
}
```

In Stored Procedure:

```
CREATE PROCEDURE getAccountsPaginated
    @braCode INT, @page INT, @pageSize INT
AS
BEGIN
    SELECT * FROM ChartOfAccounts
    WHERE braCode = @braCode
    ORDER BY accCode
    OFFSET (@page - 1) * @pageSize ROWS
    FETCH NEXT @pageSize ROWS ONLY;
END
```

6. Add TLS Encryption for Database Connection

In clsCN constructor:

```
SqlConnectionStringBuilder builder = new SqlConnectionStringBuilder(
    connectionString);

builder.Encrypt = true;
builder.TrustServerCertificate = false; // Use valid cert in production
builder.ConnectTimeout = 30;
```

Or in connection string:

```
"Server=...;Database=...;Encrypt=true;TrustServerCertificate=false;..."
```

```
...
```

6.3 Medium Priority Improvements (Priority 3)

```
...
```

7. Add Password Complexity Validation

In PasswordHelper:

```
public static ValidationResult ValidateStrength(string password)
{
    if (password.Length < 8)
        return new ValidationResult(false, "أحرف على الأقل 8");

    if (!password.Any(char.IsUpper))
        return new ValidationResult(false, "حرف كبير واحد على الأقل");

    if (!password.Any(char.IsLower))
        return new ValidationResult(false, "حرف صغير واحد على الأقل");

    if (!password.Any(char.IsDigit))
        return new ValidationResult(false, "رقم واحد على الأقل");

    if (!password.Any(c => !char.IsLetterOrDigit(c)))
        return new ValidationResult(false, "رمز خاص واحد على الأقل");

    return new ValidationResult(true, "قوة كلمة المرور: قوية");
}
```

8. Add Audit Log Retention Policy

Create scheduled job (SQL Agent):

```
CREATE PROCEDURE cleanupOldAuditLogs
```

```
AS
```

```

BEGIN
    DECLARE @cutoffDate DATETIME = DATEADD(YEAR, -1, GETDATE());

    DELETE FROM AuditLog WHERE AuditTime < @cutoffDate;
END

```

Schedule: Daily at 2:00 AM

9. Implement Business Layer Interfaces for Testability

```

public interface IUserAuthentication
{
    DataTable Login(int braCode, string userID, string password);
    void ChangePassword(int userCode, string oldPwd, string newPwd);
    bool ValidateSession(Guid token);
}

public class clsUsers : IUserAuthentication { ... }

public interface IAuditLogger
{
    void Log(string eventType, string actionName, ...);
    void LogException(Exception ex);
}

public class AuditHelper : IAuditLogger { ... }

```

...

7. ملخص المشروع | Project Summary

7.1 Statistics

Metric	Count
Total Forms (PL)	74
Business Classes (BL)	13
DAL Classes	1
Stored Procedures	50+
Database Tables	40+

| Lines of Code (approx) | 15,000+ |

7.2 Security Posture

Aspect	Status	Notes
Password Storage	✓ Strong	PBKDF2-SHA256, 100k iterations
SQL Injection	✓ Protected	Regex validation in DAL
Session Management	✓ Secure	GUID tokens, sliding expiry
Access Control	✓ Implemented	6-permission per screen
Audit Logging	✓ Active	Fire-and-forget async
Transport Security	△ Needs Work	Add TLS to DB connection
Rate Limiting	✗ Missing	Add login attempt throttling

7.3 Architecture Compliance

Principle	Status	Notes
3-Tier Separation	✓ Full	PL→BL→DAL→DB
No Business Logic in PL	✓ Compliant	All logic in BL
DAL as Single Entry	✓ Compliant	One clsCN class
Stored Procedure Usage	✓ Compliant	No inline SQL
Transaction Support	✓ Implemented	In clsCN

8. خارطة طريق التنفيذ | Implementation Roadmap

...

Phase 1: Security Hardening (Week 1-2)

- Add TLS encryption to DB connection
- Implement login rate limiting
- Add global exception handler
- Password complexity validation

Phase 2: Performance Optimization (Week 3-4)

- Implement connection pooling
- Add async session validation
- Pagination for large result sets

Phase 3: Maintainability (Week 5-6)

- Add BL interfaces for testing
- Implement audit log retention
- Document all SPs

Phase 4: Monitoring (Week 7-8)

- Add performance counters
- Implement health check endpoint
- Dashboard for key metrics

...

****Document Generated****: 2025
****Project****: IntegratedAccountsSystem
****Architecture****: WinForms 3-Tier (PL/BL/DAL)
****Version****: 1.0